

Projetando um Gerenciador de Memória Virtual

Trabalho Prático 2 - Sistemas Operacionais

Davi Lage Braga

Gabriel Santos Martins

Isaías Alves de Souza Santos

Laura Noronha Lara

Lucas Moraes Rocha Spiazzi

1 de julho de 2026

Resumo

Este trabalho apresenta o projeto e a implementação, em linguagem C, de um simulador de gerenciador de memória virtual que traduz endereços lógicos de 16 bits em endereços físicos sobre um espaço de endereçamento de 65.536 bytes. O simulador integra os principais mecanismos do subsistema de memória: a extração de página e deslocamento, a tabela de páginas, o *Translation Lookaside Buffer* (TLB) com política FIFO, a paginação por demanda a partir de um arquivo de retaguarda (*backing store*) e a substituição de páginas por LRU aproximado (algoritmo de *aging*). Ao final, são relatadas as taxas de falta de página e de acerto do TLB para dois padrões de acesso: aleatório e com localidade de referência, evidenciando o impacto da localidade sobre o desempenho da hierarquia de memória.

Sumário

1	Introdução e Fundamentação Teórica	3
2	Arquitetura e Estruturas de Dados	3
2.1	Fluxo de tradução	3
2.2	Estruturas de dados	4
3	Implementação e Decisões de Projeto	5
3.1	Como a lógica foi construída	5
3.2	Validação incremental	5
3.3	Dificuldades encontradas	5
4	Testes Realizados e Análise dos Resultados	6
4.1	Metodologia	6
4.2	Resultados	6
4.3	Análise	6
5	Conclusão	7
6	Uso de Inteligência Artificial	7
6.1	Contexto de utilização	7
6.2	Prompts utilizados	7
	Repositório	8

1 Introdução e Fundamentação Teórica

Gerenciadores de memória virtual são responsáveis por interceptar a requisição de um endereço lógico feita pelo processador e coordenar os demais componentes para localizar o dado e retornar o byte correto, gerenciando as informações mantidas em memória e em disco conforme a necessidade.

Este trabalho consiste no projeto de um gerenciador de memória virtual em C, com o objetivo de simular a forma como o hardware e o sistema operacional cooperam para dar a um processo a ilusão de dispor de um grande espaço de memória, mapeando dinamicamente o espaço de endereçamento lógico na memória física (RAM).

Fundamentação teórica: Na **paginação**, o espaço de endereçamento é dividido em blocos de tamanho fixo: as *páginas*, no espaço lógico, e os *quadros*, na memória física — ambos com 256 bytes neste projeto. Um endereço lógico é, então, interpretado como um par (*número da página, deslocamento*), em que o deslocamento indica a posição do byte dentro da página. A **tabela de páginas** armazena, para cada página, o quadro em que ela reside e se essa associação é válida. Como consultar a tabela a cada acesso seria custoso, utiliza-se o **TLB** (*Translation Lookaside Buffer*), uma memória associativa pequena e rápida que guarda as traduções mais recentes. Quando uma página referenciada não está na memória física, ocorre uma **falta de página** (*page fault*): sob **paginação por demanda**, a página é carregada do *backing store* apenas nesse momento. Como a memória física é menor que o espaço lógico, eventualmente é necessário **substituir** uma página residente; para isso emprega-se uma aproximação do algoritmo LRU (*Least Recently Used*) conhecida como **aging**, que estima o uso recente de cada página por meio de um contador atualizado periodicamente.

Organização do relatório: A Seção 2 descreve a arquitetura modular e as estruturas de dados. A Seção 3 detalha a implementação e as decisões de projeto. A Seção 4 apresenta a metodologia de testes e a análise dos resultados. A Seção 5 encerra o trabalho, seguida da documentação do uso de inteligência artificial.

2 Arquitetura e Estruturas de Dados

O projeto utiliza uma arquitetura modular, com separação entre interfaces (`include/*.h`) e implementações (`src/*.c`). Cada módulo possui uma responsabilidade única, expõe apenas as funções necessárias em seu cabeçalho e mantém o estado interno em variáveis `static`, aproximando-se do princípio de ocultação de informação. A Tabela 1 resume os módulos e suas responsabilidades.

2.1 Fluxo de tradução

Cada endereço lógico lido percorre as seguintes etapas:

1. extração do número da página e do deslocamento a partir dos 16 bits menos significativos do endereço;

Módulo	Arquivos	Implementação
Configuração	<code>config.h</code>	Constantes do sistema (tamanhos, quantidades e caminho do <i>backing store</i>).
Orquestração	<code>main.c</code>	Laço principal: lê os endereços e coordena TLB, tabela de páginas e tratamento de <i>page fault</i> .
TLB	<code>tlb.{h,c}</code>	Cache das traduções mais recentes, com política de substituição FIFO.
Tabela de páginas	<code>page_table.{h,c}</code>	Mapeamento página → quadro e metadados do LRU aproximado.
Memória física	<code>memory.{h,c}</code>	Quadros, leitura do <i>backing store</i> , <i>page fault</i> e substituição.
Estatísticas	<code>statistics.{h,c}</code>	Contadores e cálculo das taxas finais.

Tabela 1: Módulos do simulador e suas responsabilidades.

- consulta ao TLB — em caso de acerto, o quadro é obtido diretamente;
- em caso de falha no TLB, consulta à tabela de páginas — se a entrada for válida, o quadro é obtido;
- se a entrada for inválida, ocorre *page fault* e a página é carregada do *backing store* para um quadro;
- inserção da tradução no TLB e atualização dos metadados de uso (bit de referência e contador de *aging*);
- cálculo do endereço físico e leitura do byte correspondente.

2.2 Estruturas de dados

As principais estruturas que sustentam o simulador são:

- **Tabela de páginas** — vetor de 256 entradas indexado pelo número da página (acesso $O(1)$); cada entrada guarda o quadro, o bit de validade, o bit de referência e o contador de *aging* de 8 bits.
- **TLB** — vetor de 16 entradas (página, quadro e validade) acompanhado de um índice circular `fifo_next` que materializa a política FIFO; a busca é linear.
- **Memória física** — matriz `physical_memory[128][256]` de bytes sinalizados (32 KiB), acompanhada do vetor `frame_to_page[128]`, que registra qual página ocupa cada quadro (o valor -1 indica quadro livre).
- **Estatísticas** — três contadores inteiros: total de endereços traduzidos, faltas de página e acertos do TLB.
- **Backing store** — ponteiro `FILE*` para o arquivo `BACKING_STORE.bin`, tratado como arquivo de acesso aleatório.

3 Implementação e Decisões de Projeto

Por se tratar de um projeto modularizado, adotamos um plano de ataque que consistiu em implementar e concluir cada componente individualmente, verificando as dependências entre eles antes de avançar. Seguindo a ordem sugerida no enunciado, partimos da tradução básica de endereços, que descarta os bits acima dos 16 menos significativos, passamos pela leitura do *backing store* e pela tabela de páginas e, só então, integramos o tratamento de faltas de página, a substituição por LRU aproximado (*aging*) e, por último, o TLB e as estatísticas. O projeto carrega as páginas apenas sob demanda, utiliza 128 quadros de memória física e implementa o envelhecimento (*aging*) como aproximação do LRU. Essa progressão foi deliberada: cada camada só era construída sobre uma base já testada, o que manteve o código enxuto e facilitou localizar a origem de eventuais erros.

3.1 Como a lógica foi construída

Optamos por ser transparentes quanto ao processo. O projeto-base já fornecia a estrutura modular, os cabeçalhos, o *Makefile* e, sobretudo, funções esqueleto acompanhadas de comentários que descreviam exatamente o que cada trecho pendente (marcado como *TODO*) deveria fazer. Esses comentários funcionaram como uma especificação: a partir deles, a lógica de cada função foi desenvolvida com o apoio de ferramentas de inteligência artificial, adotando a abordagem orientada por especificação (*Spec-Driven Development*), na qual o problema, os requisitos e as restrições de cada trecho eram definidos com clareza antes de solicitar ou escrever o código. O detalhamento desse uso e os respectivos *prompts* constam na Seção 6.

O maior esforço do grupo, porém, não foi gerar código, e sim **entender, integrar e validar** cada peça: interpretar os comentários do template, garantir que as funções conversassem corretamente entre si e confirmar, a cada etapa, que o comportamento observado correspondia ao esperado e de certa forma, a barreira de conhecimento técnico na linguagem C foi também um fator crítico na produção do código, afinal, poucos realmente eram proeficientes nessa linguagem em nosso grupo. Além disso, as dificuldades em rodar o compilador dessa linguagem se provaram um grande desafio, já que ao contrário do Java, C possui um compilador "rústico".

3.2 Validação incremental

A verificação acompanhou o desenvolvimento passo a passo, e não apenas ao final. Após cada etapa, recompilávamos o projeto — exigindo ausência de avisos — e executávamos testes pontuais: casos pequenos em que o valor lido deveria coincidir com o deslocamento, um teste de estresse com mais páginas do que quadros disponíveis (forçando a substituição) e a execução completa sobre os arquivos de entrada. Os resultados desses testes são apresentados na Seção 4.

3.3 Dificuldades encontradas

A principal dificuldade não esteve nas funções isoladas, mas na **coerência entre as estruturas de dados**. Como a tabela de páginas, o vetor que associa quadros a páginas e o TLB descrevem, cada um à sua maneira, o mesmo estado da memória, qualquer atualização precisa ser refletida em todos eles. Enfrentamos, por exemplo, situações em que a página recém-carregada não era

registrada corretamente em todas as estruturas, o que fazia a memória parecer sempre disponível e impedia o mecanismo de substituição de entrar em ação. Depurar esse tipo de inconsistência exigiu acompanhar o percurso de um endereço de ponta a ponta e reforçou a importância de manter as três visões da memória sempre sincronizadas — especialmente ao expulsar uma página, momento em que ela deve ser invalidada tanto na tabela de páginas quanto no TLB.

4 Testes Realizados e Análise dos Resultados

4.1 Metodologia

A verificação foi realizada em múltiplas frentes:

- **Compilação limpa:** o projeto compila sem avisos com as opções `-Wall -Wextra`, tratando alertas do compilador como potenciais defeitos.
- **Correção da tradução:** como o `BACKING_STORE.bin` é gerado de modo que o byte de índice i vale $i \bmod 256$, o valor lido em um deslocamento deve coincidir com o próprio deslocamento; casos pequenos confirmaram essa propriedade.
- **Substituição sob pressão:** um teste com 256 páginas distintas (o dobro dos 128 quadros) forçou sucessivas substituições; todos os valores lidos permaneceram corretos, sem corrupção nem falhas de execução.
- **Robustez:** com entrada vazia, o programa encerra sem divisão por zero, exibindo taxas nulas.
- **Execução completa:** o simulador foi executado sobre os dois arquivos de 10.000 endereços fornecidos.

4.2 Resultados

A Tabela 2 apresenta as estatísticas obtidas para os dois padrões de acesso.

Arquivo	Endereços	Page Faults	PF Rate	TLB Hits	TLB Hit Rate
<code>addresses_random.txt</code>	10.000	4.998	0,500	627	0,063
<code>addresses_location.txt</code>	10.000	1.031	0,103	8.021	0,802

Tabela 2: Resultados das simulações para os dois arquivos de entrada.

4.3 Análise

Os resultados confirmam o comportamento teórico esperado. No arquivo de acessos aleatórios, os endereços distribuem-se uniformemente pelas 256 páginas, de modo que os 128 quadros disponíveis são insuficientes e cerca de metade dos acessos resulta em falta de página (taxa de 0,500); o TLB, com apenas 16 entradas, raramente contém a página seguinte, produzindo uma taxa de acerto de somente 0,063. Já no arquivo com localidade de referência, os acessos concentram-se em páginas próximas, o que permite reaproveitar tanto as páginas já residentes quanto as traduções

armazenadas no TLB: a taxa de faltas cai para 0,103 e a taxa de acerto do TLB sobe para 0,802. A diferença mostra na prática o impacto do **princípio da localidade** sobre o desempenho da hierarquia de memória.

5 Conclusão

Este trabalho consistiu na implementação de um simulador de gerência de memória virtual em C, capaz de traduzir endereços lógicos em endereços físicos. Ao longo do desenvolvimento, foram exercitados os mecanismos centrais do subsistema de memória de um sistema operacional: a decomposição do endereço lógico em número de página e deslocamento, a tabela de páginas como mapeamento completo, o TLB como cache de traduções, a paginação por demanda apoiada em uma memória de retaguarda e uma política de substituição baseada no LRU aproximado.

Os objetivos propostos foram cumpridos: o simulador compila sem avisos, executa corretamente sobre os arquivos fornecidos e produz resultados coerentes com o comportamento esperado de um gerenciador de memória virtual, como demonstrado pela análise da localidade de referência na Seção 4. Além do resultado em si, o trabalho ajudou a entender melhor como os conceitos vistos na teoria se traduzem em estruturas de dados e algoritmos concretos, e quais compromissos de projeto estão por trás de cada escolha.

6 Uso de Inteligência Artificial

Esta seção descreve como usamos ferramentas de inteligência artificial durante o trabalho. Seguimos a abordagem orientada por especificação (*Spec-Driven Development*): antes de pedir qualquer coisa à ferramenta, definíamos com clareza o problema, os requisitos e as restrições de cada tarefa. Em todas as etapas, o grupo revisou, integrou e validou o que foi gerado, ou seja, a IA serviu de apoio, não de substituta do nosso trabalho.

6.1 Contexto de utilização

Usamos a IA em quatro frentes: planejar o desenvolvimento a partir dos módulos do projeto, documentar o código-fonte com *docstrings*, montar a estrutura inicial do relatório em LaTeX e transcrever para esse formato o conteúdo que o grupo redigiu, conferindo a consistência das informações.

6.2 Prompts utilizados

Os *prompts* abaixo reproduzem, já em redação revisada, as instruções fornecidas à ferramenta:

1. **Planejamento do desenvolvimento:** “Elabore um plano de desenvolvimento estruturado a partir dos módulos do projeto, apresentando as etapas de forma didática e com instruções claras e objetivas.”
2. **Documentação do código-fonte:** “Documente a finalidade de cada método do código-fonte por meio de *docstrings* concisas e, ao mesmo tempo, de fácil compreensão.”

3. **Estrutura do relatório:** “Gere a estrutura inicial de um relatório técnico em LaTeX no diretório `/report`.”
4. **Transcrição e verificação de consistência:** “Tomando como referência o documento em PDF com o conteúdo do relatório elaborado pelo grupo, transcreva-o para o arquivo LaTeX, assegurando a consistência e a integridade das informações.”

Repositório

O código-fonte completo está disponível publicamente em:

https://github.com/catmaitachi/vm_manager

Referências

- [1] SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Fundamentos de Sistemas Operacionais*. Rio de Janeiro: LTC.